RESUMEN TÉCNICO: CURSO DE HTML Y CSS

1. Fundamentos de la Web

Para comprender el desarrollo web, es necesario entender la arquitectura básica de internet y el flujo de la información:

- **w.w.w (World Wide Web):** Es la red global de contenidos interconectados.
- **Servidor:** Es el equipo donde se aloja el archivo de la página web; su función principal es recibir las peticiones de las computadoras y entregar el contenido.
- **Navegador:** Es el software que actúa como intérprete. Conecta nuestra computadora con el servidor mediante un enlace y nos permite leer las páginas web, similar a como un reproductor de video abre archivos de video.

Flujo de Petición:

1. Nuestra computadora envía una petición al servidor a través de una URL.
2. El servidor recibe la petición y busca el archivo.
3. El navegador recibe el código e interpreta las instrucciones para mostrarlo visualmente, ya que todo el código escrito está hecho para ser interpretado por él.

2. HTML: Definición y Estructura

HTML (HyperText Markup Language) es un lenguaje de marcado de hipertexto, no de programación.

- **Función:** Se encarga de armar la estructura de la página web mediante el uso de etiquetas (tags) que agrupan y etiquetan el contenido.
- **Lenguaje Interpretado:** HTML tomó texto plano y lo presenta de forma diferente, pero requiere de un intérprete (el navegador) para mostrar el resultado visual.
- **Archivo Base:** El archivo principal debe tener la extensión **.html** y comúnmente se nombra **index.html**, ya que el navegador busca este nombre por defecto para mostrar el contenido inicial.

Sintaxis y Estructura del Documento

El código se organiza mediante **elementos definidos por etiquetas**: `<tag>Contenido</tag>` (se abre, se coloca el contenido y se cierra).

Estructura estándar (HTML5):

- `<!DOCTYPE html>`: Define la **versión de HTML** que se está utilizando.
- `<html> </html>`: Etiqueta raíz; define que **todo lo que está dentro es parte de la página web.**

- `<head> </head>`: Contiene la información técnica de la página que **no es visible** para el usuario (metadatos).
    - `<title> </title>`: Define el **título que aparece en la pestaña** o ventana del navegador.
- `<body> </body>`Contiene toda la **parte visual** de la página web.

3. Elementos de Contenido (Body)

Estos son los **elementos visibles que estructuran la información** dentro del `<body>`:

Encabezados (Headings)

Sirven para **títulos y subtítulos**. La jerarquía va del 1 al 6:

- `<h1>`: El título más importante. Se utiliza **una sola vez** por página.
- `<h2>`: Subtítulos importantes.
- `<h3> a <h6>`: A medida que aumenta el número, el encabezado es más pequeño y tiene menos importancia semántica.

Texto y Listas

- `<p> </p>`: Define **párrafos** de texto.
- **Listas Ordenadas (`<ol> </ol>`):** Ordered List. Muestra los **elementos numerados**.
- **Listas Desordenadas (`<ul> </ul>`):** Unordered List. Muestra los **elementos con viñetas**.
- **Ítem de lista (`<li> </li>`):** List Item. Etiqueta obligatoria que debe ir dentro de `<ol>` o `<ul>` para definir cada punto de la lista.

---

4. Enlaces y Multimedia

Diferencia clave: Una **Página web es un archivo individual**, mientras que un **Sitio web es un conjunto de páginas** unidas por enlaces.

Enlaces (Links)

Se crean con la etiqueta de ancla `<a> </a>`.

- **href="ruta":** Atributo obligatorio que indica el destino. Si se usa `https://` es un enlace externo; si no, busca localmente en las carpetas.
- **target="_blank":** Abre el **enlace en una nueva ventana** o pestaña.
- **target="_self":** Abre el **enlace en la misma página** (comportamiento por defecto).

- **title="texto":** Muestra un pequeño **texto descriptivo al pasar el mouse** sobre el enlace.

Imágenes

Se insertan con la etiqueta `<img>`. Nota: **Esta etiqueta no necesita cierre.**

- **src="archivo":** Indica la **ruta de la imagen.**
- **alt="texto": Texto alternativo** que se muestra si la imagen no carga por algún error.

Tipos de Rutas:

- **Absolutas:** La dirección completa (no importa desde dónde se acceda).
- **Relativas:** La dirección depende de la ubicación del archivo actual.

---

5. Formularios e Interacción

Los formularios permiten recolectar datos del usuario. Se envuelven en la etiqueta `<form> </form>`.

El elemento `<input>`

Es el **campo donde el usuario interactúa**. Sus atributos principales son:

Tipos de input (type):

- **type="text":** Texto simple.
- **type="email":** Valida que el texto tenga formato de correo electrónico.
- **type="password":** Oculta los caracteres con asteriscos o puntos.
- **type="color":** Selector de colores.
- **type="checkbox":** Casilla para tildar opciones.
- **type="radio":** Permite seleccionar una única opción de un grupo.
- **type="file":** Permite subir archivos.
- **type="submit":** Botón para enviar el formulario.

Atributos de validación y control:

- **required:** Hace que el campo sea obligatorio.
- **placeholder="texto":** Muestra un texto de ayuda que desaparece al escribir.
- **value="":** Define un valor inicial o el valor que será enviado al servidor.
- **name="":** Asigna un nombre identificador al input.
- **minlength="" / maxlength="":** Define la longitud mínima y máxima de caracteres.

---

6. CSS (Hojas de Estilo)

CSS se utiliza para declarar y modificar las propiedades estándar que los navegadores asignan a los elementos HTML (tamaño, color, bordes, etc.).

Busca entender la relación entre las propiedades.

Reglas de Estilo

Para aplicar CSS, seguimos una estructura lógica:

1. **Selector:** Indica a qué elemento HTML (etiqueta) hace referencia.
2. **Bloque de declaraciones:** Contiene las propiedades que queremos modificar y los nuevos valores que les asignaremos.

7. Implementación de CSS

Para mantener el código limpio, no debemos mezclar lenguajes. Aunque existe la etiqueta `<style>`, lo profesional es tener el CSS en un archivo separado.

Vinculación de archivo externo: Utilizamos la etiqueta `<link>` dentro del `<head>` del HTML para conectar nuestra hoja de estilos.

- `<link rel="stylesheet" href="estilos.css">`:
  - `rel="stylesheet"`: Indica que el archivo es una hoja de estilo.
  - `href="..."`: Indica la ruta del archivo CSS.

8. Tipos de Selectores

Los selectores nos permiten indicar al navegador exactamente qué elemento queremos modificar.

1. **Por Elemento:** Selecciona todas las etiquetas de ese tipo (ej. `p`, `h1`, `div`). También se usa `<span>` para seleccionar partes muy puntuales de texto.
2. **Por Clase (`.`):** Agrupa varios elementos diferentes bajo un mismo estilo.
   - En HTML: `class="nombre-clase"`
   - En CSS: `.nombre-clase { ... }`.
3. **Por Identificador (`#`):** Identifica a un elemento **ÚNICO**.
   - En HTML: `id="nombre-unico"`
   - En CSS: `#nombre-unico { ... }`.

9. Propiedades de Texto y Tipografía

Fuentes (Fonts)

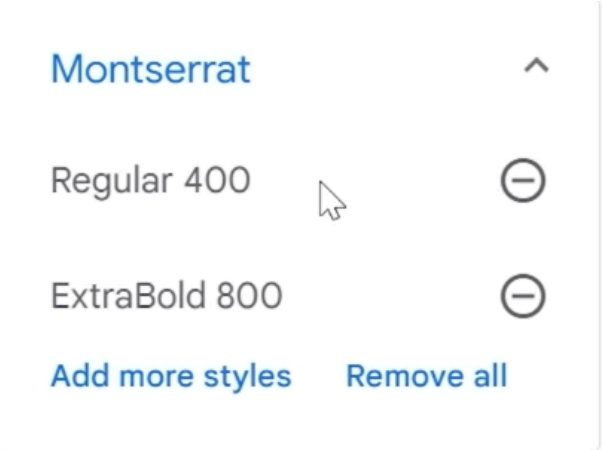
- `font-family`: Define la tipografía. Se pueden poner varias separadas por coma como respaldo (ej. `tahoma`, `sans-serif`).
- `font-size`: Tamaño de la letra (ej. `16px`).
- `font-weight`: Grosor de la letra.
    - Values: `bold` (negrita), `bolder` (más grueso), o números (ej. `900`),.
- `font-style`: Estilo de la letra.
    - `italic` (cursiva), `oblique` (inclinada) o `normal`,.

Estilizado de Texto

- `color`: Color del texto. Se recomienda usar códigos absolutos (Hex, RGB) en lugar de nombres (`"red"`).
- `text-align`: Alineación (`left`, `right`, `center`, `justify`).
- `text-decoration`: Decoraciones como `underline` (subrayado), `line-through` (tachado) o `none` (para quitar subrayado a enlaces).
- `text-transform`: Cambia mayúsculas/minúsculas.
    - `uppercase` (todo mayúscula), `lowercase` (todo minúscula), `capitalize` (primera letra mayúscula),.
- `line-height`: Altura entre líneas de texto.
- `letter-spacing`: Espacio entre caracteres.
- `text-shadow`: Aplica sombras al texto.
- `word-wrap`: Controla cómo se rompen las palabras al final de la línea (`break-word` corta respetando sílabas/palabras; `break-all` corta bruscamente).

Tipografías Externas

1. **Google Fonts:** Se busca la fuente, se copia el `<link>` y se pega en el `<head>`.




```
<link rel="preconnect" href="http
s://fonts.gstatic.com" crossorigin
>
<link href="https://fonts.googleap
is.com/css2?family=Montserrat:wght
@400;800&display=swap" rel="styles
heet">
```

```
<head>
    <title>Propiedades de Texto</title>
    <link rel="stylesheet" href="styles.css">
    <link rel="preconnect" href="https://fonts.
    googleapis.com">
<link rel="preconnect" href="https://fonts.
gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Montserrat:wght@400;800&display=swap"
rel="stylesheet">
</head>
```

2. **@font-face:** Permite alojar la fuente en nuestro propio servidor para mayor control y compatibilidad. Se define el nombre y la ruta del archivo (`.ttf` u otros),.

```css
@font-face {
    font-family:"Propio";
    src:url("Montserrat_Alternates/MontserratAlternates-Regular.ttf")
format("truetype");
    font-weight: 900
}
```

```css
p {
    color:blueviolet;
    font-family:Propio;
    font-weight:900;
    font-style:italic
}
```

10. El Modelo de Caja (Box Model)

En CSS, todo elemento es una caja. Esta caja se compone de cuatro partes, desde adentro hacia afuera:

1. **Content (Contenido):** El texto, imagen o información real (`width` y `height`).
2. **Padding (Relleno):** Espacio transparente entre el contenido y el borde,.
3. **Border (Borde):** La línea que delimita la caja.
4. **Margin (Margen):** Espacio vacío fuera de la caja que separa a los elementos entre sí.

Propiedades de la Caja

- `width / height`: Ancho y alto del contenido.
- `background`: Color o imagen de fondo de la caja.

Padding y Margin (Shorthand): Se pueden definir los 4 lados en una sola línea siguiendo el sentido de las agujas del reloj: `padding: 10px 5px 4px 10px;` (Top, Right, Bottom, Left).

- **Colapso de márgenes:** Si dos cajas están una sobre otra (verticalmente), sus márgenes no se suman; se aplica el margen mayor, o se superponen (ej. `10px` de una y `10px` de otra resultan en `10px` de separación total, no `20px`).
- `margin: auto`: Centra una caja horizontalmente respecto a su contenedor.

Bordes (border)

Sintaxis abreviada: `border: width style color;`

- Ejemplo: `border: 3px solid black;`.

- `border-radius`: Redondea las esquinas. Puede ser un valor general o específico por esquina,.

Box Sizing

Define cómo el navegador calcula el tamaño total del elemento.

- `box-sizing: content-box` (Default): Si agregas padding o borde, el tamaño total de la caja aumenta, deformando el diseño.
- `box-sizing: border-box`: (Recomendado) El tamaño total se mantiene fijo. Si agregas padding o borde, el contenido interno se achica para compensar, evitando que la caja crezca.

11. Colores y Unidades de Medida

Formatos de Color

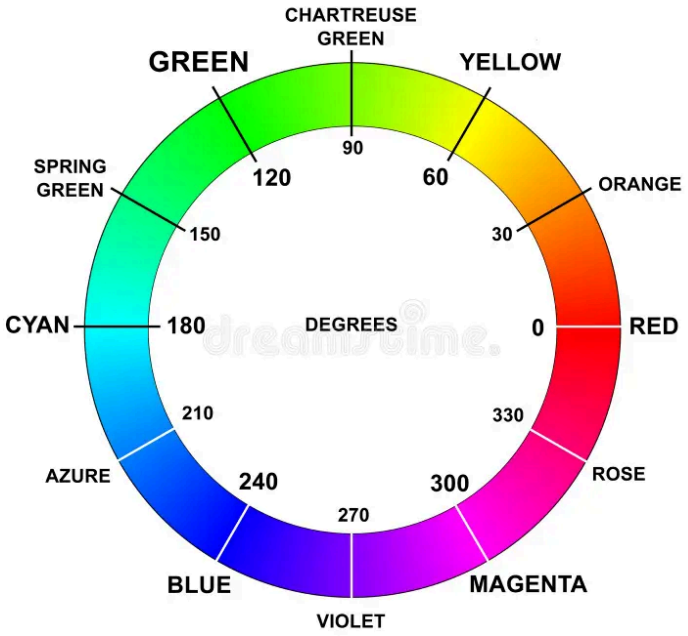
- **Hexadecimal:** Código de 6 caracteres (ej. `#00ff00`). Los primeros dos son rojo, los siguientes verde, los últimos azul. Puede abreviarse (ej. `#0f0`).
- **RGB:** `rgb(rojo, verde, azul)`. Valores del 0 al 255.
- **RGBA:** Igual que RGB pero agrega un canal Alpha (transparencia) al final (0 es transparente, 1 es sólido).
- **HSL/HSLA:** `hsl(tono, saturación%, luminosidad%)`. Útil para manejar paletas de colores intuitivas.

11. Colores y Unidades de Medida

Formatos de Color

- **Hexadecimal:** Código de 6 caracteres (ej. `#00ff00`). Los primeros dos son rojo, los siguientes verde, los últimos azul. Puede abreviarse (ej. `#0f0`).
- **RGB:** `rgb(rojo, verde, azul)`. Valores del 0 al 255.
- **RGBA:** Igual que RGB pero agrega un canal Alpha (transparencia) al final (0 es transparente, 1 es sólido).
- **HSL/HSLA:** `hsl(tue, saturación%, luminosidad%)`. Útil para manejar paletas de colores intuitivas:

  Tue:



Unidades de Medida

Existen dos tipos principales según tus anotaciones:

A. Medidas Absolutas

Son medidas fijas que se utilizan cuando necesitamos precisión exacta (como en impresión), pero no se recomiendan para pantallas responsivas (celulares) porque no se adaptan bien.

- `px` (píxeles): La mínima área de luz en la pantalla. Es la más utilizada en web para detalles precisos.
- `cm` (centímetros) / `mm` (milímetros): Unidades del sistema métrico.
- `in` (pulgadas): Medida física (inches).
- `pt` (puntos): Unidad tipográfica estándar, generalmente no se usa en web.

B. Medidas Relativas

Estas dependen del tamaño de otros elementos y sí se adaptan a distintos dispositivos.

- `%` (Porcentaje): Relativo al tamaño del contenedor padre.
- `em`: Relativo al tamaño de fuente del elemento padre. Si el padre no tiene, busca al abuelo, etc..
- `rem`: Relativo al tamaño de fuente del elemento raíz (`html`). Es más predecible que `em`.

- `vh` (Viewport Height): Porcentaje de la altura visible de la pantalla (`100vh = 100% de la altura`).
- `vw` (Viewport Width): Porcentaje del ancho visible de la pantalla.
- `vmax / vmin`: Toma el valor máximo o mínimo entre el alto y el ancho de la pantalla.
- `ch`: Equivale al ancho del número `"0"` de la fuente actual.
- `ex`: Equivale a la altura de la letra `"x"` minúscula de la fuente actual.

Recomendación de uso (Resumen)

- Para tipografía: Usar `em` y `rem`.
- Para layout (estructura): Usar `%`, `vh`, `vw`.
- Para detalles precisos fijos: Usar `px`.

Recursos Web Recomendados

Herramientas útiles mencionadas para trabajar con colores:

- **HTML Color Codes (https://htmlcolorcodes.com/):** Para encontrar códigos de colores en todos los formatos (RGB, Hex, HSL).
- **Generador de Paletas (https://paletadecolores.com.ar/):** Ayuda a crear combinaciones de colores armónicas.

12. Propiedades de Fondo (Backgrounds)

CSS permite un control total sobre los fondos de las cajas.

Propiedades Individuales

- `background-image: url("ruta")`: Establece una imagen.
- `background-size`:
  - `cover`: Cubre todo el fondo (puede recortar la imagen para llenar espacios).
  - `contain`: Muestra la imagen entera al menos una vez.
  - `100%`: Fuerza el ancho total.
- `background-position`: Ubicación (ej. `center`, `top`, o coordenadas `x` y `y`).
- `background-repeat`: `no-repeat` evita el efecto mosaico.
- `background-attachment: fixed`: Efecto paralaje; la imagen queda fija al hacer scroll.

Shorthand (Atajo)

```
background: color url position / size repeat attachment;
```

- Ejemplo: `background: green url("img.jpg") center / cover no-repeat fixed;`.

Gradientes

- `linear-gradient(dirección, color1, color2)`: Transición lineal.
- `radial-gradient()`: Desde el centro hacia afuera (circular).
- `conic-gradient()`: Rotación alrededor del centro.

13. Efectos Visuales y Sombras

- `box-shadow`: Sombra para cajas.
  - Sintaxis: `offset-x offset-y blur spread color`.
  - Ejemplo: `10px 10px 5px 0px black`.
- `text-shadow`: Sombra exclusiva para tipografía.
- `filter: drop-shadow(...)`: Genera una sombra que respeta la transparencia de imágenes PNG (silueta perfecta).

14. Propiedad Display

Define el comportamiento de visualización.

- `display: inline-block`: Permite que los elementos se sitúen uno al lado del otro (como texto) pero respetando sus dimensiones de ancho y alto (como bloques).

15. HTML Avanzado: Metadatos y SEO

Información técnica alojada en el `<head>` para navegadores y motores de búsqueda (SEO).

- **Comentarios:** Código invisible para el usuario: `<!-- Comentario -->`.
- **Favicon:** El icono de la pestaña: `<link rel="icon" href="icono.png" type="image/png">`.

Meta Tags Fundamentales

- `charset="UTF-8"`: Codificación universal de caracteres.
- `viewport`: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`. Esencial para que la web sea responsive en móviles.

SEO

- `description`: Resumen en los resultados de búsqueda.
- `keywords`: Palabras clave para los buscadores.
- `author`: Creador del sitio.

Control de Robots (Google)

`<meta name="robots" content="...">`.

- `nofollow`: No seguir enlaces.
- `nosnippet`: No mostrar fragmentos de texto.
- `noimageindex`: No indexar las imágenes.

Open Graph (Redes Sociales)

Define cómo se ve la web al compartirla en redes (WhatsApp, Facebook).

- `og:title`, `og:image`, `og:description`.

16. Elementos de Formulario Avanzados

Textarea (`<textarea>`)

A diferencia del `<input>` normal, este elemento permite escribir múltiples líneas de texto (párrafos).

- **Sintaxis HTML:** `<textarea placeholder="Escriba su mensaje aquí"></textarea>`.

Atributos HTML Clave

- `rows="n"`: Define la altura inicial en cantidad de líneas de texto visibles.
- `maxlength="n"`: Establece el límite máximo de caracteres permitidos.
- `wrap`: Controla cómo se comportan los saltos de línea automáticos.
- `readonly`: Modo "solo lectura". El usuario ve el texto pero no puede editarlo; sin embargo, el dato **SÍ** se envía al servidor.
- `disabled`: Campo desactivado. El usuario no puede interactuar y el dato **NO** se envía al servidor.

Estilos y Control CSS para Textarea

El `textarea` es un elemento particular porque el usuario puede cambiar su tamaño manualmente desde la esquina. Para evitar que esto rompa nuestro diseño, usamos las siguientes propiedades:

1. Control de Redimensionamiento (`resize`)

Define si el usuario tiene permiso para cambiar el tamaño de la caja.

- `resize: vertical`: Solo puede modificar la altura (Recomendado).
- `resize: horizontal`: Solo puede modificar el ancho.
- `resize: none`: Bloquea el cambio de tamaño totalmente (fijo).

2. Límites de Dimensiones (Evitar roturas)

Si permitimos que el usuario agrande la caja, debemos poner límites para que no tape otros elementos o se salga de la pantalla:

- `min-height / min-width`: Define el tamaño mínimo para asegurar que el texto sea legible.
- `max-height / max-width`: Define el tamaño máximo permitido (ej. `max-width: 100%` para que no se salga del contenedor padre).

3. Ajuste Automático (`form-sizing`)

- `form-sizing: content`: (Propiedad moderna) Hace que el textarea crezca automáticamente a medida que el usuario escribe, eliminando la necesidad de barras de desplazamiento.

17. Organización y Etiquetas de Formulario

Para mejorar la accesibilidad y la experiencia del usuario, HTML ofrece etiquetas específicas para organizar y etiquetar los campos.

Etiquetas (`<label>`)

Es la forma correcta de ponerle nombre a un input. Mejora la accesibilidad porque permite que los lectores de pantalla identifiquen el campo. Además, al hacer clic en el texto del label, el navegador hace foco automáticamente en el input.

- **Forma 1 (Vinculación por ID):** Se usa el atributo `for` en el label y el `id` en el input.
    - `<label for="nombre">Nombre:</label> <input id="nombre">`
- **Forma 2 (Anidación):** Se coloca el input dentro del label.

    - `<label>Email: <input type="email"></label>`

Agrupación (`<fieldset>` y `<legend>`)

Permiten agrupar visual y semánticamente varios controles dentro de un formulario (muy útil para secciones como "Datos Personales" o "Datos de Pago").

- `<fieldset>`: Es el contenedor o caja que agrupa los inputs.
- `<legend>`: Define el título o leyenda del grupo.

Listas Desplegables y Opciones

- **Select (`<select>`):** Crea un menú desplegable clásico donde el usuario elige una opción.
    1. Las opciones se definen con `<option value="valor">Texto Visible</option>`.
    2. El atributo `value` es la información interna que llega al servidor; el texto es lo que ve el usuario.
- **Datalist (`<datalist>`):** Es un híbrido entre un input y un select. Permite al usuario escribir libremente o seleccionar una sugerencia predefinida (autocompletado).
    1. Se crea un input con el atributo lista: `<input list="milista">`.
    2. Se define el datalist con ese ID: `<datalist id="milista">`.
    3. Dentro se ponen las opciones (`<option value="...">`).

18. Elementos Interactivos Nativos

HTML5 incluye elementos que permiten interactividad sin necesidad de JavaScript ni CSS complejo.

Detalles y Resumen (`<details>` y `<summary>`)

Crea un acordeón o menú desplegable nativo.

- **Funcionamiento:** Por defecto muestra solo el contenido de `<summary>`. Al hacer clic, se despliega el resto del contenido dentro de `<details>`.
- **Estilos CSS:** Podemos personalizar su apariencia según si está abierto o cerrado usando el selector de atributo `details[open]`.

19. Enlaces Avanzados y Seguridad

Más allá de la navegación básica, los enlaces tienen atributos críticos para la seguridad y el rendimiento.

Tipos de Enlaces

- **Anclas (Internos):** Para navegar a una sección específica dentro de la misma página.
    - Se pone un `id` al destino (`<h1 id="seccion1">`) y se enlaza con numeral (`<a href="#seccion1">`).
- **Descargas:** El atributo `download` fuerza al navegador a descargar el archivo en lugar de abrirlo.
    - `<a href="archivo.pdf" download>Descargar</a>`.

Atributos de Seguridad (`rel`)

Cuando usamos `target="_blank"` (abrir en nueva pestaña), exponemos una vulnerabilidad de seguridad (el sitio nuevo puede tener acceso al objeto `window.opener` del sitio original). Para evitarlo, usamos el atributo `rel`:

1. `rel="noopener"`: Bloquea el acceso al `window.opener` (seguridad crítica).
2. `rel="noreferrer"`: No envía información de referencia (el sitio de destino no sabrá desde qué web llegaste).
3. `rel="nofollow"`: Indica a los motores de búsqueda (Google) que no rastreen ese enlace ni le pasen autoridad de tu sitio.

- **Uso recomendado:** `<a href="..." target="_blank" rel="noopener noreferrer">`.

20. Tablas de Datos

Se utilizan exclusivamente para mostrar datos tabulares (filas y columnas), no para maquetar el sitio. Son muy útiles en el diseño de emails (newsletters).

Estructura Básica

- `<table>`: Contenedor principal.
- `<tr>` (Table Row): Define una fila.
- `<th>` (Table Header): Celda de encabezado (negrita y centrada por defecto).
- `<td>` (Table Data): Celda de datos estándar.

Organización Semántica

- `<thead>`: Agrupa la cabecera de la tabla.
- `<tbody>`: Agrupa el cuerpo con los datos.
- **Fusión de Celdas:** El atributo `colspan="n"` permite que una celda ocupe el espacio de `n` columnas.

21. Multimedia Avanzada: Video y Audio

Etiquetas nativas para reproducción de medios.

Etiqueta `<video>`

Soporta atributos booleanos (se activan solo con nombrarlos):

- `controls`: Muestra los controles de reproducción (pausa, volumen).
- `autoplay`: Inicia el video automáticamente al cargar.
    - **Nota:** La mayoría de los navegadores bloquean el autoplay si el video tiene sonido. Para que funcione, se debe combinar con `muted`.

- `muted`: Silencia el video.
- `loop`: Reproduce el video en bucle infinito.

Subtítulos (`<track>`)

Permite añadir archivos de texto para subtítulos o descripciones.

- `<track kind="captions" src="subs.vtt" srclang="es" label="Español">`.

22. Optimización y Semántica del Texto

Lazy Loading (Carga Diferida)

Es una técnica de optimización de rendimiento. Hace que las imágenes o iframes se carguen solo cuando el usuario hace scroll y se acerca a ellas, ahorrando datos y acelerando la carga inicial.

- **Sintaxis:** `<img src="foto.jpg" loading="lazy">`.

Etiquetas Obsoletas vs. Modernas

Muchas etiquetas antiguas (`<font>`, `<center>`, `<big>`, `<strike>`, `<tt>`) han quedado obsoletas porque su función era puramente visual (ahora trabajo de CSS).

- **Reemplazos:** Usar `<s>` o `<del>` para tachado, `<code>` para código, y CSS para alineación y fuentes.

Semántica de Texto (SEO)

Aunque visualmente parezcan iguales, los buscadores las interpretan distinto:

- `<b>` vs `<strong>`: Ambas ponen negrita. `<b>` es solo visual; `<strong>` indica importancia fuerte para el SEO.
- `<i>` vs `<em>`: Ambas ponen cursiva. `<i>` es visual; `<em>` indica énfasis en la pronunciación o contexto.

23. HTML Semántico y Estructura

Utilizar las etiquetas correctas ayuda a Google a entender qué es cada parte de tu web (SEO) y mejora la mantenibilidad del código.

Etiquetas Estructurales

- `<header>`: Cabecera del sitio o sección (logos, menús).
- `<nav>`: Bloque de enlaces de navegación principal.
- `<main>`: Contenido principal y único de la página.
- `<section>`: Sección temática del contenido.
- `<article>`: Contenido independiente y autónomo (ej. un post de blog).
- `<aside>`: Contenido secundario o relacionado (barras laterales).
- `<footer>`: Pie de página (créditos, contacto, legales).

24. Accesibilidad Web (A11Y)

El objetivo es que la web sea utilizable por todos, incluyendo personas con discapacidad que usan lectores de pantalla (Screen Readers).

Funcionamiento

El navegador crea el DOM visual y, paralelamente, un Árbol de Accesibilidad. Los lectores de pantalla leen este segundo árbol. Si nuestro HTML está mal construido, el árbol de accesibilidad será incorrecto.

Herramientas ARIA (Accessible Rich Internet Applications)

Cuando el HTML nativo no es suficiente, usamos atributos ARIA:

- `aria-label="..."`: Provee una etiqueta textual invisible en pantalla pero audible para el lector (fundamental para botones que son solo íconos).
- `role="..."`: Define explícitamente qué función cumple un elemento (ej. si usas un `div` como botón, debes ponerle `role="button"` para que el lector sepa que es clicable).

¡Ah, ya entiendo el problema! Lo que te está pasando es que cuando copias desde acá, mi respuesta se lleva "códigos ocultos" (como los bloques oscuros que uso para mostrar el código de programación), y cuando los pegás en tu PDF, Word o Google Docs, la estructura choca y se rompe todo.

Para que te quede perfecto y con la misma estructura visual de tu PDF (títulos en negrita, subtítulos y viñetas clásicas, sin cuadros de código molestos), te lo reescribí usando el formato de texto enriquecido estándar.

Copiá el texto desde acá abajo, pegalo en tu archivo y debería encajar a la perfección con el estilo que ya traías:

25. Selectores Avanzados en CSS

Los selectores nos permiten apuntar a los elementos HTML a los que queremos dar estilo.

- **Selector de atributo:** Selecciona elementos que contengan un atributo específico. (Ej. `img[src="imagen.png"] { }`)
- **Selección descendiente:** Apunta a un elemento que está dentro de otro. (Ej. `ul li { }`)
- **Selector por hijo directo (>):** Apunta únicamente a los elementos que son descendientes directos. (Ej. `li > em { }`)
- **Selector de hermano adyacente (+):** Selecciona al elemento hermano que viene después del primer elemento que yo asigno. (Ej. `h1 + p { }`)
- **Selector de hermanos generales (~):** Selecciona todos los elementos hermanos que vienen después del elemento que yo asigno. (Ej. `h1 ~ p { }`)

26. Herencia, Especificidad y Cascada

Muchas de las propiedades que asignan a elementos, muchas veces se heredan a los hijos o elementos que forman parte de ese elemento padre, como lo que sucede con el body.

- **color: inherit:** Esto hace que el elemento aplique la propiedad que va a heredar de su elemento padre.
- **color: initial:** Va a tener el valor inicial, el valor que le da el navegador por defecto, sin heredar el valor del padre.
- **color: unset:** Si el valor por defecto era inherit lo vuelve a inherit pero si por defecto era otro, lo deja en initial.
- **color: revert:** Restaura la propiedad al estilo base del navegador.

Especificidad y Cascada

- **Selector Specificity:** Nos especifica la especificidad del elemento. Los elementos que mayor orden de especificidad tengan, van a ser los que apliquen la propiedad.

Jerarquía de especificidad:

1. !IMPORTANT (ej. `color: blue !important`)
2. atributo style="" (darle style dentro de la etiqueta)
3. etiqueta `<style></style>`
4. id
5. class
6. elemento común

Si yo combino estas formas puedo obtener elementos que tengan mayor especificidad todavía.

27. Pseudo-clases y Pseudo-elementos

Las pseudo clases también agregan especificidad. **Pseudo-Clases:** Nos permite seleccionar un estado específico de un elemento que seleccionemos.

- **h1:hover:** Seleccionamos el h1 cuando el mouse esté por encima.
- **h1:active:** Cuando le damos click.
- **li:first-child:** Selecciona el primero.
- **li:last-child:** Selecciona el último.
- **li:nth-child(N):** En N especificamos el número del elemento que queremos modificar.
- **li:nth-child(2n):** Modificó intercalados, es decir en este caso, de cada 2 elementos modifico 1.
- **li:nth-of-type(5):** Cuento solo los elementos que son del tipo li.
- **li:not(.excluir):** Todos los elementos que tengan la clase excluir, van a ser excluidos de las propiedades que yo les asigne.
- **div:empty:** Se le asigna la propiedad solo si el elemento está vacío.
- **:root:** Selecciona al elemento raíz.
- **input[type="checkbox"]:checked:** Cuando el checkbox esté chequeado, le aplico alguna propiedad.
- **a:link:** Enlaces que no fueron visitados.
- **a:visited:** Los enlaces que fueron visitadas.

Pseudoclases de agrupación y condicionales (is, where, has):

- **section:is(.elemento1, .elemento2):** Agrupo elementos, selecciono varios elementos y le aplico alguna propiedad. Útil cuando usamos selector descendiente.
- **section:where(.elemento1, .elemento2):** “Where” no cambia la especificidad en cambio, “is” si cambia la especificidad, es la única diferencia.
- **section:has(.elemento1):** Verifica si el elemento section tiene el elemento 1, en caso de que si lo tenga, aplicó alguna propiedad.
- **ul:has(+div):** En caso de que a “ul”, le siga un div, modificamos el ul.

- **label:has(input:valid):** Modificar el label en caso de que el input sea válido.

Pseudoelementos: Representa una parte específica de un elemento. Se representan con dos puntos.

- **h1::first-letter:** Selecciona la primer letra.
- **div::first-line:** Selecciona la primera linea.
- **div::selection:** Cuando lo seleccionamos, es decir apretamos el click y arrastramos el mouse para seleccionarlo.
- **input::placeholder:** Modifica el texto de ayuda.
- **li::marker:** Las viñetas de las listas.
- **li::before / li::after:** Agrego un elemento antes o despues del elemento asignado, pero el navegador no lo muestra, porque en realidad no es un elemento.

28. Metodología BEM (Block Element Modifier)

Nos permite reutilizar código. Estandarizamos código, creamos como bases o códigos comunes que siempre aparecen en los elementos para ahorrar tiempo.

- **Bloque:** Componente independiente que no depende de nadie para existir. Al bloque lo nombró con su funcionalidad (ej. `class="lista"`).
- **Elemento:** Pedazo del bloque, elemento de ese bloque, que fuera de ese bloque pierde el sentido (ej. `class="lista__item"`).
- **Modificadores:** Formas de modificar los elementos y los bloques. Agrego una segunda clase al elemento (ej. `class="lista__item item--importante"`).

29. Display y Posicionamiento Display

- **display: block:** El elemento se vuelve un bloque, ocupan todo el ancho disponible y empieza una linea abajo. El padding y el margin van a funcionar bien.
- **display: inline:** Elementos de bloques los volvemos en línea. El alto y el ancho no importan, es por eso que el padding y el margin solo funcionan horizontalmente.
- **display: inline-block:** Es una mezcla entre un elemento en línea y en bloque a la vez. Le puedo dar un alto y un ancho. Muy util para barras de navegacion.
- **display: none:** El elemento no ocupa espacio en la página, esta, existe, pero no se representa visualmente.

Posición Relativa y Absoluta

- **position: relative:** Mantiene la posición original en la página, pero deja que la movamos (`top: 50px`, `left: 50px`). El lugar de la caja en la página queda reservado. La propiedad top es más importante que bottom, y left más importante que right.
- **z-index: 10:** Nos permite mover los elementos y ponerlos adelantes de otros en el eje Z. El valor más alto va a estar por delante.
- **position: absolute:** Deja de ocupar lugar en la página, el elemento está, pero no ocupa lugar. Ahora se mueve en relación a la página entera o a su contenedor.
- **position: fixed:** Siempre pero siempre se va a posicionar en relación a la página entera, y además cuando scrolleamos en la página, la caja se queda quieta.
- **position: sticky:** Funciona como un position relative, pero cuando le específico los píxeles, la caja cuando se encuentre a esos pixeles del borde de la página, se va a empezar a comportar como un position fixed.

30. Etiqueta Dialog y Desbordamiento (Overflow)
Etiqueta dialog

Muy útil para ventanas modales. Con la etiqueta dialog, puedo cerrar esta ventana usando el `method="dialog"` dentro de un formulario y utilizando un botón tipo submit.

- **dialog::backdrop:** Se usa con `<dialog>` para estilizar el fondo oscuro que aparece detrás cuando el dialog está abierto como modal (ej. `background: rgba(0, 0, 0, 0.5);`).

Overflow Cómo se maneja el contenido que sobresale de su contenedor.

- **overflow: visible:** Propiedad por defecto, lo que sobresale del contenedor que se vea.
- **overflow: hidden:** Todo lo que sale del contenedor, se oculta, no se ve.
- **overflow: scroll:** Podemos scrollear pero se genera un scroll abajo que es muy poco estético. Se puede usar overflow-x e y.
- **overflow: auto:** De los mejores valores, una vez que me pase del contenido, se muestra el scroll, sino no aparece.
- **overflow: clip:** Funciona como el hidden, pero hidden reserva espacio para las barras de scroll, en cambio clip no.

31. Control de Texto, Dimensiones Dinámicas y Contorno Dimensiones Dinámicas

- **width: min-content:** Encoge la caja lo máximo posible sin desbordar el contenido. En texto, el ancho de la caja será exactamente del tamaño de la palabra más larga.
- **width: max-content:** La caja se estirará infinitamente para que todo el contenido ocupe una sola línea sin realizar saltos de carro.
- **width: fit-content:** Es una combinación; actúa como max-content expandiéndose, pero si alcanza el límite de su contenedor padre, hace saltos de línea normales y respeta el borde.

Propiedades que controlan el flujo del texto

- **white-space: nowrap:** Que no hayan saltos de línea automáticos.
- **white-space: pre-wrap:** Aplica los saltos de línea que yo pongo en mi html y además los automáticos cuando se sale el contenido.
- **text-overflow: clip:** Se recorta el texto.
- **text-overflow: ellipsis:** Cuando se desborda el texto, se agregan 3 puntitos.
- **word-break: break-all:** Recorta las palabras.
- **text-wrap: balance:** Balancea el texto para que todos encajen de alguna manera más linda. Útil para títulos.
- **text-wrap: pretty:** Útil para párrafos en vez de títulos, evitan que queden palabras sueltas al final o al principio del texto (se las conoce como viudas).
- **initial-letter: 2 3:** Para la primera letra. Primer valor es la cantidad de líneas que va a ocupar hacia arriba, el segundo es desde que línea lo va a ocupar.

Contorno (outline) Entre el margen y los bordes. Tiene la característica de que no aumenta el tamaño de la caja, no ocupa espacio, a diferencia de los bordes. Es una guía para el usuario.

- **outline: 5px solid blue:** Atajo para usar width, style y color juntos.
- **outline-offset: 10px:** Cuantos pixeles me separó desde el borde hasta el inicio del contorno.
- **Accesibilidad:** Si un input no tiene outline, tenemos problemas de accesibilidad. Lo correcto es utilizar `input:focus-visible { outline: none; }` y darle otro estilo de enfoque visual.

32. Multimedia Flexible e Imágenes Optimizadas Object Fit

Cómo encaja el elemento una vez que yo defino el alto y el ancho del contenedor (para imágenes).

- **object-fit: fill:** Se adapta, ajusta la escala.
- **object-fit: contain:** Hace que la imagen entre por lo menos una vez en el contenedor, sin deformarse.
- **object-fit: cover:** Se adapta, pero modifica la imagen recortando.
- **object-fit: none:** La resolución original de la imagen se mantiene.
- **aspect-ratio: 16/9:** Se le da al contenedor del video para mantener la relación de aspecto sin importar cómo se agrande o achique la pantalla.

SrcSet y Sizes

- **Sintaxis en img:** `<img src="foto.png" srcset="foto-small.png 400w" sizes="(max-width:400px) 300px">`. Le recomiendo al navegador qué imagen cargar según la pantalla.
- **Etiqueta `<picture>`:** Mezcla de srcset y sizes pero mejor usado. Se usan etiquetas `<source media="(max-width:400px)" srcset="img-small.png">`. Hay que tener cuidado de que las imágenes están ordenadas de menor a mayor.

Clip Path Para hacer un recorte, todo lo que esté fuera del recorte no se ve. Normalmente se utilizan páginas externas (como CSS clip-path maker).

- **clip-path: circle(50% at 100px 100px):** Movemos el punto del medio de la imagen.
- **clip-path: polygon(...):** Para cortes exactos y polígonos.

33. FlexBox

Módulo de css, para que las cajas sean flexibles y su contenido también.

- **display: flex:** Los elementos dentro de la caja van a ser flexibles, la caja contenedora va a ser el flex-box, y los elementos los flex item.
- **flex-direction:** row (van de izquierda a derecha), row-reverse, column (arriba hacia abajo), column-reverse.
- **flex-wrap: wrap:** En vez de que los elementos se achiquen para que entren, los pasa debajo.
- **flex-flow: row wrap:** Combina el flex-wrap y el flex-direction.

Alineación Eje Principal (justify-content) Alinea los ítems en el eje principal en dirección del texto.

- **justify-content:** start, end, center.
- **justify-content: space-between:** Te aleja todos los elementos entre si a la misma distancia, poniendo el primero al inicio y el ultimo al final.
- **justify-content: space-evenly:** Distribuye todo de manera equitativamente, ordena todo a la misma distancia entre los elementos y con el contenedor.

Alineación Eje Cruzado (align-items y align-content)

- **align-items:** stretch (los elementos se estiran para llenar el eje), start, end, center, baseline (se alinean según la linea base del texto).
- **align-content:** Funciona igual que justify-content pero para alinear cuando tenemos muchas lineas en el eje cruzado.
- **gap: 20px:** A todos los elementos dentro del flexbox lo separamos, como hacer un margin general.

Flex Basis, Shrink y Grow (En los hijos)

- **order: 1:** Le doy órdenes a los elementos, mayor orden va más a la derecha.
- **flex-grow: 1:** Del espacio disponible puede ocupar algo, el elemento puede crecer.
- **flex-shrink: 1:** Hasta donde se van a encoger los elementos si falta lugar.
- **flex-basis: 50px:** Dejo de usar widht y height, uso flex-basis para darle un tamaño base.
- **flex: 0 1 auto:** Junta grow, shrink y basis en una sola propiedad.
- **align-self: center:** Afecta a un solo item, en vez de usar align-items que afecta a todos.

34. GRID

Cuando trabajamos con grillas o contenidos que van a cambiar y van a ir en dos direcciones, es muy útil utilizar GRID. No tenemos una sola dimensión como flexbox, podemos trabajar con filas y columnas a la vez.

- **Grid Container:** Elemento padre. Se declara con display: grid. Por defecto, al crearlo, todos los elementos del contenedor se vuelven filas (se crea una grilla de 1 sola columna).
- **Grid Items:** Los elementos dentro del padre.
- **Grid Track:** Líneas que separan, puede ser una fila o una columna entera.
- **Grid Cell:** Unidad básica donde se puede poner un grid item (una celda).

Tamaño de Filas y Columnas (Grid Explícito) Son los elementos a los cuales nosotros les definimos los tamaños explícitamente.

- **grid-template-columns: 200px 200px 100px:** Agregamos el tamaño de las columnas. Cada valor corresponde a una columna (acá tendríamos 3 columnas).
- **grid-template-rows: 100px 200px:** Define el tamaño de las filas.
- **Valor auto:** grid-template-columns: auto auto 100px. Analiza que el contenido siempre entre, y lo que sobra por fuera del contenido mínimo, lo reparte entre las cajas con el valor "auto".
- **Valor fr (Fracción):** grid-template-columns: 1fr 1fr 1fr. Significa una fracción del espacio disponible. Reparte todo en partes iguales.
- **Función repeat():** grid-template-columns: repeat(5, 1fr). Forma de repetir los tamaños o patrones. Acá crea 5 columnas de 1fr.
- **Función minmax():** grid-template-columns: minmax(300px, 500px). El elemento se agranda desde 300px como mínimo hasta 500px como máximo.
- **Grid Gap:** Debido a que el margin no colapsa en grid, usamos gap para separar celdas. gap: 25px 5px (Primer parámetro es para las filas/ROW, el segundo para las columnas/COLUMN). Si pongo uno solo (gap: 20px), aplica a los dos.

Grid Dinámico e Implícito

- **Grid Implícito:** Son elementos que no están definidos por nosotros. Grid crea filas nuevas automáticamente para adaptar estos elementos que sobran.
- **grid-auto-rows: 100px:** Define el tamaño fijo que tendrán esas filas nuevas que se crean solas.
- **grid-auto-flow: dense:** Soluciona el problema de que cuando cambio las resoluciones me queden espacios en blanco en la grilla.
- **Grid Dinámico (Responsive sin media queries):**
    - **auto-fit:** repeat(auto-fit, minmax(150px, 1fr)). Verifica si en la resolución entra otra columna para agregar y "adapta". Necesita elementos para crear estas columnas.
    - **auto-fill:** repeat(auto-fill, minmax(150px, 1fr)). "Rellena". Crea columnas por más de que no haya elementos para llenarlas.

Ubicación de Elementos (Para los hijos)

- **Ubicación manual por líneas:** grid-column-start: 1 (define en qué línea empieza el elemento). grid-row-end: 3 (define en qué fila termina).
- **Atajo span:** grid-column-end: span 2. Esto me dice que el elemento va a terminar abarcando 2 columnas más.
- **Atajo Start y End juntos:** grid-column: 1 / span 2 (Empieza en la línea 1 y abarca 2 lugares).

Grid Area (Diseño Visual) Partes del grid que ocupan más de una celda. Podemos organizar las áreas de toda la grilla de la siguiente manera en el elemento padre: grid-template-areas: "header header header" "nav main aside" "footer footer footer"

- Luego, a cada elemento (hijo) le aplicó su área correspondiente. Ej. en el CSS del header pongo: grid-area: header;.

Alineación con Grid

- **Eje Horizontal:**
    - **justify-items:** start, center, end, stretch. Alinea los ítems adentro de las celdas de la grilla (Se aplica al grid container).
    - **justify-content:** center. Mueve todas las celdas juntas en el eje horizontal.
    - **justify-self:** Para mover un solo ítem específico dentro de su celda (Se aplica al grid item).
- **Eje Vertical:** Funciona exactamente igual pero verticalmente. Usamos align-items, align-content y align-self.

Subgrid

Cuando yo creo una grilla dentro de otra grilla, esos elementos no tienen en cuenta a la grilla padre. Subgrid es la forma de hacer que los hijos de un grid-item también formen parte y respeten la grilla principal.

- Debemos agregar a la subgrilla esto: grid-template-columns: subgrid;.

35. Diseño Responsivo, Variables y Funciones CSS

Media Queries y Diseño Responsivo

- **Mobile First:** Forma de pensar el desarrollo, en donde primero creamos la página web adaptada a celular y luego en dispositivos más grandes.
- **@media screen and (max-width:500px) { ... }:** Aplicó un estilo cuando la pantalla cumpla la condición.
- **Feature Queries (@supports):** Nos permite detectar si el navegador del usuario soporta una característica de CSS antes de aplicarla (ej. `@supports (display: grid)`).
- **Container Queries (@container):** En vez de medir la pantalla entera, mide el tamaño del contenedor padre.

  Se usa

  `container-type: inline-size` y `container-name: main`.

  Luego `@container main (max-width:600px) { ... }`.

Unidades del Viewport

- **vh / vw:** La altura y el ancho del viewport total.

- **dvh / dvw (Dynamic View):** Esto es muy útil, ya que en celu el viewport es muy distinto porque no tiene en cuenta la barra de navegación. DVH soluciona este problema directamente. De ahora en más utilizamos siempre el DVH y DVW.

Variables en CSS

- **Estructura:**

  En el `:root` ponemos `--color-principal: #4CAF50;`.

  En el elemento la aplicamos con `color: var(--color-principal);`.

- **@property:** Otra manera de crear variables.

  ```
  syntax: "<color>",

  inherits: false,

  initial-value: black.
  ```

Funciones Matemáticas y Comparación

- **min(600px, 800px):** Me devuelve siempre el más chico para adaptarse.
- **max(600px, 800px):** Devuelve el máximo.
- **clamp(400px, 100%, 1400px):** Le doy un valor mínimo primero, luego un valor ideal (100%), y luego un valor máximo.
- **calc(100% - 100px):** Función que nos permite sumar, restar, multiplicar y dividir en CSS.

36. Filtros y Transformaciones (Sección Avanzada)
Filtros (filter y backdrop-filter)

Funciones de Filtro que se utilizan en la función filter:

- **filter: drop-shadow(0px 0px 15px color):** Agrega una sombra al elemento, a diferencia de box-shadow respeta transparencias.
- **filter: blur(10px):** Desenfoque.
- **filter: brightness(0.5):** Establecer brillo.
- **filter: grayscale(1):** Se convierte completamente en gris.
- **filter: invert(1):** Invierte los colores (de 0 a 1).
- **backdrop-filter:** Se aplica el filtro, no al contenido, sino al fondo del contenido, pudiendo darle un desenfoque por ejemplo.

Color Mix

- **color: color-mix(in srgb, red 30%, blue 30%):** Esto nos permite combinar colores definiendo los porcentajes.

Transform

- **scale(1.5):** Aumenta su tamaño, aumenta la escala.
- **skew(20deg):** Rota el elemento tantos grados, pero no la gira, la transforma, mueve los bordes.
- **rotate(20deg):** Directamente gira el elemento.
- **translate(100px, 200px):** Traslada el elemento, el primer valor en el eje X y el segundo en el eje Y.

37. Animaciones y Scroll Propiedades de Scroll

- **scroll-behavior: smooth:** La velocidad de subida y bajada del scroll, es más suave, se ve solo cuando hago un click en un link y bajo en la página.
- **scrollbar-color: black red:** Debemos darle dos colores si o si, uno es el de la barra y el otro el fondo.

Transiciones Necesitan un evento disparador.

- **transition-property: color.** `transition-duration: 1s`. `transition-delay: 3s`.
- **transition-timing-function: ease-in** (empieza lento y termina brusco).
- **transition: left 1s ease 2s:** Atajo que junta todo (propiedad, tiempo, velocidad, y delay).

Animaciones (@keyframes) Proceso que quiere generar una ilusión de movimiento. Pueden ser infinitas y automáticas.

- **@keyframes nombre { 0% { opacity: 0; } 100% { opacity: 1; } }**
- **animation-name: nombre.** `animation-duration: 2s`.
- **animation-fill-mode:** forwards (cuando termina se queda en el estado final), both (empieza con el estado inicial y termina con el final).
- **animation-play-state: paused:** La animación se pausa, muy útil para hover.

Animaciones con Scroll y View

- **animation-timeline: scroll():** El timeline va a estar definido por el scroll, a medida que yo avanzo con el scroll, el elemento va cambiando.
- **animation-timeline: view():** El elemento va a ejecutar una animación, si el elemento es visible o no en pantalla.

- **animation-range-start: cover 20%:** Para que la animación empiece después, por ejemplo cuando el elemento esté al 20% de la pantalla.

Accesibilidad

- **@media (prefers-reduce-motion:reduce):** Para las personas que tengan activado esta opción en su navegador, qué son las personas que tienen epilepsia, se le agrega `animation: none`.

38. Emmet y Web Hosting Emmet

Funciona con abreviaciones, convierte estas abreviaciones en código, para facilitar la escritura.

- **>:** Con este símbolo es meterme dentro de una etiqueta para crear algo (ej. `ul > li`).
- ***:** Puedo especificar la cantidad de elementos que quiero crear (ej. `ul>li*4 > a.link`).
- **CSS:** `m10` me crea un margen de 10 px.

Web Hosting Alojamiento web, servicio que nos da un espacio para almacenar información. Nuestra página web va a estar almacenada en un servidor externo.

- **Hosting Compartido:** El más común de todos. Un solo servidor contiene muchísimos sitios web. Es la opción más económica.
- **VPS:** Servidor privado virtual, utiliza una tecnología de virtualización y separa el servidor en muchas partes, cada parte actúa como un servidor independiente.
- **Hosting Dedicado:** Un servidor dedicado para un solo cliente. Ideal para empresas grandes y con mucha demanda.
- **Cloud Hosting:** Distribuir todo el tráfico en una red de servidores. Ideal para demandas inestables.

Recomendaciones para hosting

- **Soporte las 24hs, Alto rendimiento, Confiabilidad (99% del tiempo en línea), Back Ups.**
- **HTTPS:** Protocolo que usa tu navegador con un SSL, es decir, más seguro todavía para el tráfico de datos.
- **CDN:** Almacenar las copias de contenido de manera distribuida en todos los servidores para que todo sea más óptimo.